

Unit 5 - Memory Game

Assignment Overview

The Memory Game is a classic puzzle game that challenges players to test and improve their memory skills. Players flip over cards to reveal symbols and try to find matching pairs. In this assignment, you will create a digital version of a Memory Matching Game, along with creative enhancements to make gameplay more engaging and interactive.

Basic Game Requirements

Core Game Mechanics:

Implement a memory matching game with the following features:

- The game board consists of an even number of cards arranged in a grid (e.g., 4x4, 6x6), 10 different set of images (all images are original).
- All cards start face down.
- The player flips over two cards at a time.
- If the cards match, they remain face up.
- If the cards do not match, they flip back over after a short delay.
- The game continues until all pairs are matched.
- Track and display:
 - Number of moves (or attempts)
 - Time elapsed

Program Requirements

General Requirements

- Follow the IDEA/ALTER framework as discussed in class.
- Use object-oriented programming for card objects and game logic.
- Store classes (e.g., Card, GameBoard, Player) in separate files from the main game loop.
- Use original assets for card designs and the player character to get full max.

Development Phases

1. Phase 1: Basic Game Implementation

- Create the game board with face-down cards.
- Implement card flipping functionality.
- Add matching logic to check for pairs.
- Track the number of moves.
- Implement a timer that starts when the first card is flipped.

- End the game when all pairs are found.

2. Phase 2: Enhancements

Add the following features in order:

Introduction Screen:

- Create an opening screen displayed before the game begins.
- Include:
 - Game title
 - Instructions on how to play
 - Controls
- Add a **Start button** that begins the game.
- Ensure the game does not start until the player interacts with this screen.

Mouse-Controlled Character (Cursor Replacement):

- Replace the default mouse cursor with a **custom character or sprite**.
- The character must:
 - Follow the mouse movement on the screen
 - Visually represent the player (e.g., hand, avatar, icon)
- Mouse clicks should be performed through this character to interact with cards and sound effects.
- Add simple animation.

Advanced features:

- Introduce special cards with unique effects when matched, such as:
 - Reveal additional cards temporarily
 - Shuffle remaining cards
 - Grant bonus points or reduce moves
- The player must complete the game before time runs out.
- Display remaining time on screen.

Shuffle / Replay Button:

- Add a visible **Shuffle/Play Again button** on the screen.
- When clicked, it must:
 - Reset all cards to face down
 - Shuffle the positions of all cards
 - Reset the score/move counter
 - Reset and restart the timer
- Ensure the game returns to its initial playable state without restarting the program.

Scoring System:

- Award points based on performance:
 - Fewer moves = higher score
 - Faster completion = bonus points
- Optional penalties for incorrect matches.

3. Phase 3: Aesthetics and Enhancements

- Add sound effects for flipping, matching, and game completion.

Programming and Style Guidelines

1. Documentation:

- Include a header docstring explaining your game and enhancements.
- Use clear variable names and meaningful comments.

2. Code Style:

- Follow all style rules posted..
- Name the main file `main.py`.
- Use functions, classes, and modular design effectively.
- Ensure clean, readable, and well-organized code (25 lines per methods, 30 lines per functions, including the code comment).

Submission Requirements

1. Submit your code in a zip folder with your name.
2. Include all original assets used in your game.
3. Ensure the game runs without errors and meets all requirements.